Expense Tracker — watsonx Challenge Submission

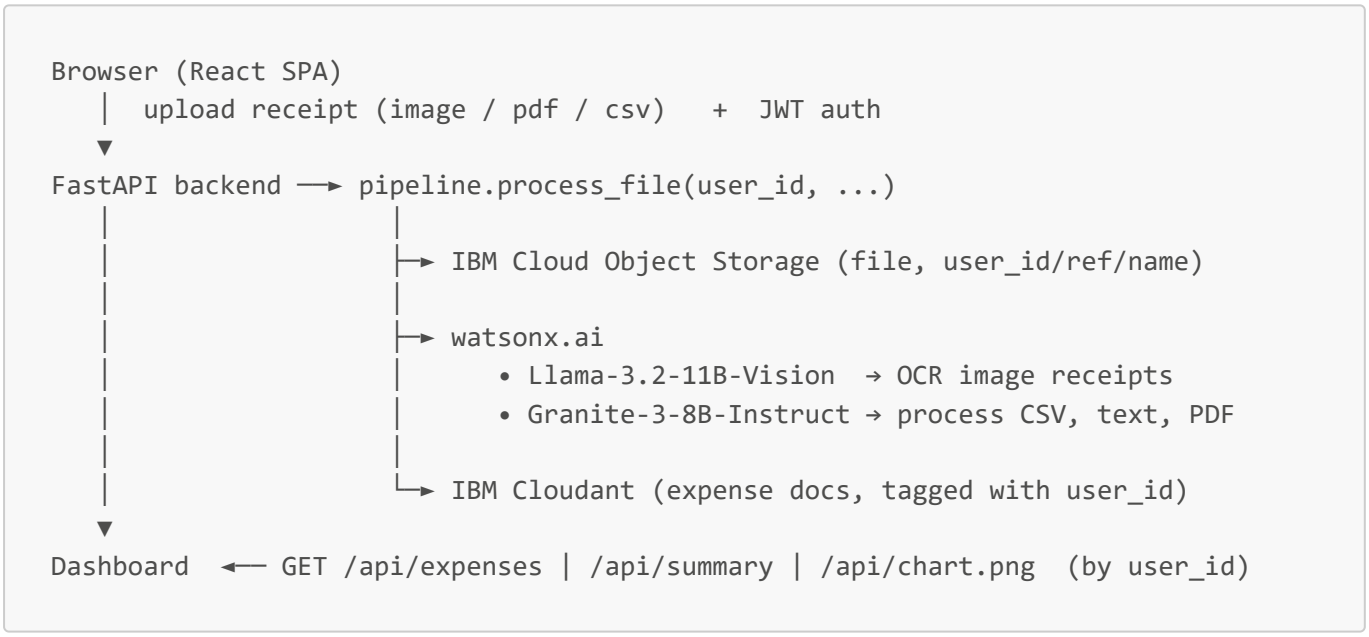
1. One-line pitch

A multi-user expense tracker that turns any receipt (image, PDF, or CSV) into categorized, queryable expenses using an **all-IBM** stack: **watsonx.ai** for vision OCR + categorization, **IBM Cloudant** for data, and **IBM Cloud Object Storage** for raw files.

2. Why it matters

Manual expense entry is tedious and error-prone. This project lets a user drop a receipt into a browser and instantly get structured, categorized expense rows on a dashboard — no manual typing, no spreadsheets. It began as a single-inbox Gmail agent and was migrated into a real multi-tenant web product where each user only ever sees their own data.

3. Architecture (all-IBM)



Two ingest paths share one processing core:

- **Web upload (primary):** browser → `/api/upload` → `pipeline.process_file`.
- **Gmail poller (secondary, retained):** cron → `run_once()` → same core.

IBM services used

Service	Role
watsonx.ai — Llama-3.2-11B-Vision	OCR of image receipts into structured lines
watsonx.ai — Granite-3-8B-Instruct	Normalize + categorize CSV / text-PDF rows
IBM Cloudant	Per-user expense documents (Mango index on `user_id`,`date`)
IBM Cloud Object Storage	Raw uploaded files, namespaced per user

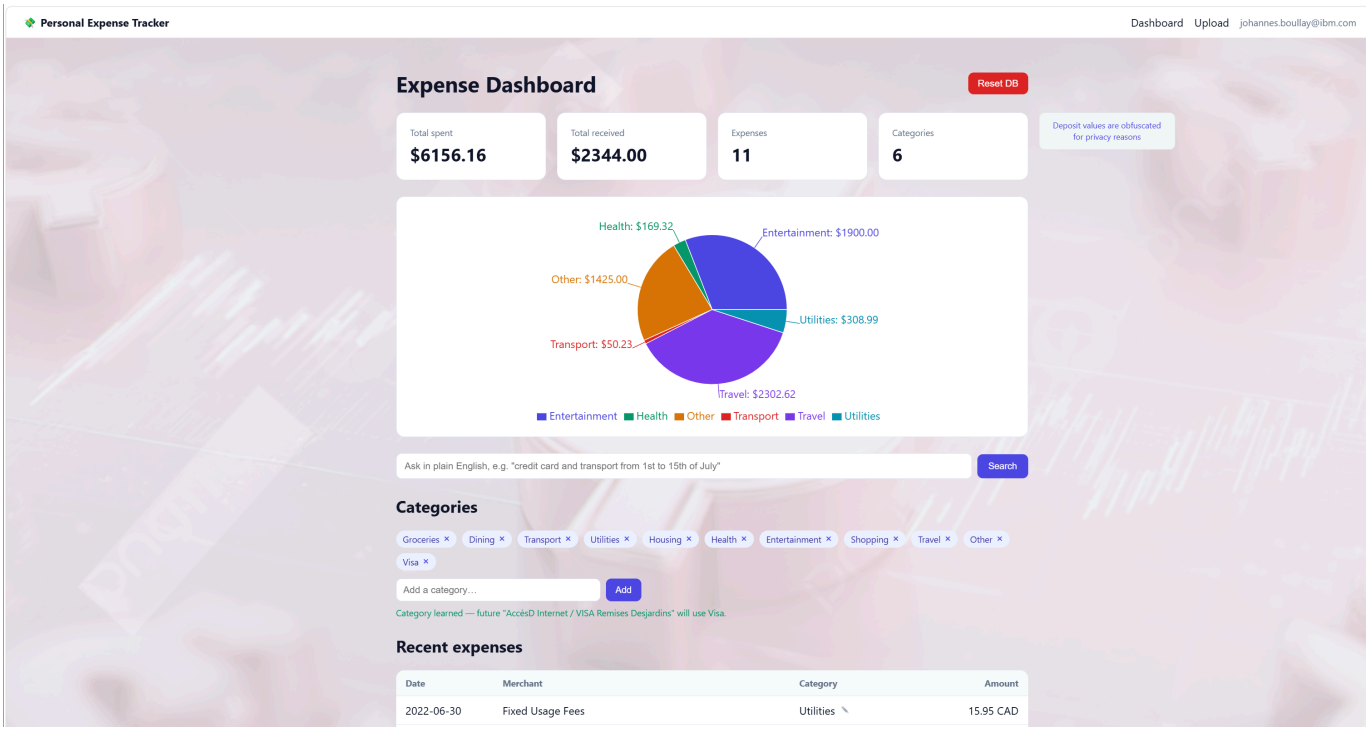
Service	Role
IBM Code Engine (planned)	Single-container hosting (API + SPA)

4. Demo script (5 minutes)

Run locally (single origin, exactly what the container would serve):

```
cd web/frontend && npm install && npm run build && cd ../..
uvicorn app:app --port 8000 --app-dir web/backend
# open http://localhost:8000
```

Screenshot:



Then walk through:

1. **Register** a new account (email + password) → JWT issued, redirected in.
2. **Upload** a receipt image on the Upload page → watsonx.ai Llama-Vision OCRs it; extracted, categorized rows appear immediately.
3. **Upload** a CSV / bank-statement PDF → Granite normalizes and categorizes every line.
4. **Dashboard** shows summary cards, a category pie chart, and the expense table — all for the logged-in user only.
5. **Natural-language query:** type a plain-English search such as "groceries from July 1st to 15th" into the dashboard search box. `POST /api/query` sends the text to watsonx.ai Granite (`watsonx_client.parse_query()`), which returns a structured JSON filter (`categories`, `date_from`, `date_to`); the backend applies it and returns a filtered summary + rows.
6. **Re-categorization with learning:** change an expense's category inline (`PATCH /api/expenses/{id}`). The choice is saved as a per-user merchant rule (`store.upsert_category_rule`), back-applied to existing expenses from the same merchant, and applied automatically to future uploads in `process_file()`.

7. **Multi-tenant proof:** register a *second* user and show their expense list is empty / disjoint — no cross-user data leakage.
8. **Idempotency:** re-upload the same file → no duplicate rows (deterministic `sha256(user_id:source_ref:line_index)` doc id).

5. Deployment status — deploy-ready

The app is **container-ready and deploy-ready**; a live IBM Code Engine URL is not included because the available IBM Cloud account is a **Lite (free) plan**, and Code Engine requires a Pay-As-You-Go account. This is a billing limitation, not a technical gap. Everything needed to deploy is in the repo:

- `web/Dockerfile` — multi-stage build (Node builds the SPA; Python serves API + SPA on one port). Verified to mirror local single-origin behavior.
- `web/README.md` — exact IBM Code Engine commands using `--build-source` (a remote/cloud build, so no local Docker daemon is required).
- `.dockerignore` — keeps secrets and build junk out of the image.

6. What's verified

- Register / login issue and accept JWTs (bcrypt password hashing).
- Authenticated upload runs the full watsonx.ai pipeline and returns extracted, categorized expenses.
- `/api/expenses`, `/api/summary`, `/api/chart.png` return **only** the caller's data; a second user sees a disjoint, empty set.
- Natural-language query: `POST /api/query` parses plain English via Granite (`watsonx_client.parse_query()`) into a `{categories, date_from, date_to}` filter and returns a scoped, filtered summary + rows.
- Re-categorization with learning: `PATCH /api/expenses/{id}` updates the category, persists a per-user merchant rule, back-applies it to existing expenses, and applies it to future uploads.
- Robust LLM JSON parsing: recovers rows even when the model output is truncated.
- No-Docker parity test: `python web/backend/test_web_smoke.py`.

7. Repo map

- `agent/` — shared AI core: `router`, `parsers`, `watsonx_client`, `schema`, `store`, `chart`, `pipeline` (the reusable engine, unchanged in spirit).
- `web/backend/` — FastAPI app: `app`, `auth`, `users`, `deps`, `routes_upload`, `routes_expenses`.
- `web/frontend/` — React + Vite SPA (Login, Upload, Dashboard).
- `web/Dockerfile`, `.dockerignore`, `web/README.md` — deployment.
- `Expense-tracker-web.md` — full phased migration plan (W0–W3).

8. Note on model endpoints

The watsonx.ai models used (Llama-3.2-11B-Vision, Granite-3-8B-Instruct) run against shared public endpoints on the Lite plan. Response latency reflects that shared capacity and would improve under a dedicated or reserved deployment.